

Kocaeli Üniversitesi
Yazılım Mühendisliği Bölümü

Programlama 2

Dönem Projesi Raporu

Snake (Yılan) Oyunu

C Programlama Dili ve SDL3 ile 2D Oyun Geliştirme

Öğrenci Adı Soyadı: İsa Koç

Öğrenci No: 210229047

May 30, 2026

Contents

1	Giriş	2
2	Kullanılan Teknolojiler ve Araçlar	3
2.1	C Programlama Dili	3
2.2	SDL3 (Simple DirectMedia Layer 3)	3
2.3	SDL3_ttf Kütüphanesi	3
3	Sistem Tasarımı ve Veri Yapıları	4
3.1	Oyun Alanı (Grid System) ve Ölçeklendirme	4
3.2	Temel Veri Yapıları	4
4	Uygulama ve Algoritma Detayları	5
4.1	Ana Oyun Döngüsü (The Game Loop)	5
4.2	Kuyruk Kaydırma ve Hareket Algoritması	5
4.3	Zamanlayıcı (Timer) Entegrasyonu	5
4.4	Çarpışma Tespit Algoritmaları (Collision Detection)	5
4.5	Rastgele Yem Üretimi (Food Spawning)	5
5	Kullanıcı Arayüzü ve Kaynak Yönetimi	6
5.1	Oyun Durumu Yönetimi	6
5.2	Bellek Temizliği (Memory Management)	6
6	Sonuç ve Değerlendirme	7
6.1	Gelecek Geliştirmeler (Future Work)	7

1 Giriş

Bu rapor, yazılım mühendisliği temel prensipleri doğrultusunda, C programlama dili ve SDL3 (Simple DirectMedia Layer 3) kütüphanesi kullanılarak tasarlanan ve geliştirilen klasik Snake (Yılan) oyununun mimarisini, yapılandırma aşamalarını ve uygulama sürecini kapsamlı bir şekilde ele almaktadır.

Oyun geliştirme süreçleri, yalnızca görsel bir çıktı üretmenin ötesinde; bellek yönetimi, veri yapıları, algoritmik düşünme ve donanım kaynaklarının verimli kullanılması gibi bilgisayar bilimlerinin temel yetkinliklerini gerektirir. Bu projenin birincil amacı, teorik programlama bilgilerini pratik bir uygulamaya dönüştürerek oyun döngüsü (game loop) mimarisini, gerçek zamanlı olay yönetimini (event handling) ve 2D grafik render işlemlerini uygulamalı olarak deneyimlemektir.

Geliştirilen kod mimarisinde, oyun alanı sabit boyutlu hücrelere bölünmüş ve yılanın gövdesi ile yem pozisyonları koordinat tabanlı veri yapıları üzerinden takip edilmiştir. Sistem, zamanlayıcı tabanlı bir yapı üzerine inşa edilmiş olup, işlemci hızından bağımsız, tutarlı bir oyun deneyimi sunmayı hedefler. Duvar ve gövde çarpışma (collision) algoritmaları koda entegre edilmiş, kullanıcı arayüzü tarafında ise anlık skor takibi ve durum yönlendirmeleri işlenmiştir.

Geliştirme ortamı olarak CachyOS (Arch tabanlı Linux dağıtımı) tercih edilmiş, derleyici olarak GCC (GNU Compiler Collection) ve metin editörü olarak Neovim kullanılarak terminal odaklı, yüksek performanslı bir iş akışı benimsenmiştir.

2 Kullanılan Teknolojiler ve Araçlar

2.1 C Programlama Dili

Projenin temel geliştirme dili olarak C seçilmiştir. C dilinin donanıma yakın seviyede çalışabilmesi, işaretçiler (pointers) aracılığıyla doğrudan bellek erişimi sunması ve düşük çalışma zamanı (runtime) yüküne sahip olması, onu performans kritik oyun uygulamaları için ideal kılmaktadır. Oyundaki yılanın kuyruk yapısı, C dilinin sunduğu bellek bloklarının ardışık yönetimi mantığıyla tasarlanmıştır.

2.2 SDL3 (Simple DirectMedia Layer 3)

Oyunun grafik, pencere ve giriş birimleri yönetimi SDL3 kütüphanesi ile sağlanmıştır. Sektör standardı olan SDL2'nin modern donanımlar ve güncel grafik API'leri (Vulkan, DirectX 12, Metal vb.) için tamamen yeniden tasarlanmış hali olan SDL3, işletim sistemine özgü karmaşık kodlar yazmak yerine, platform bağımsız (cross-platform) bir geliştirme imkanı sunar. Projede `SDL_CreateWindowAndRenderer` fonksiyonu kullanılarak donanım hızlandırmalı bir çizim motoru entegre edilmiştir.

2.3 SDL3_ttf Kütüphanesi

Oyun içindeki metinlerin (Skor göstergesi ve Oyun Bitiş ekranı) dinamik olarak oluşturulması için resmi bir eklenti olan `SDL3_ttf` kullanılmıştır. Bu eklenti, TrueType (.ttf) formatındaki yazı tiplerini okuyarak, bellek üzerinde önce bir yüzeye (`SDL_Surface`), ardından da GPU tarafından işlenebilecek bir dokuya (`SDL_Texture`) dönüştürerek ekrana yansıtır.

3 Sistem Tasarımı ve Veri Yapıları

3.1 Oyun Alanı (Grid System) ve Ölçeklendirme

Oyun, piksel tabanlı serbest bir hareket yerine ızgara (grid) tabanlı bir hareket mekaniğine sahiptir. Pencere boyutu 640×480 piksel olarak belirlenmiş ve `CELL_SIZE` sabiti ile her bir oyun hücresi 20×20 piksel olarak tanımlanmıştır. Bu matematiksel oranlama sonucunda oyun alanı, 32×24 hücrelik mantıksal bir matrise dönüştürülmüştür. Bu yaklaşım, çarpışma algılamasını ve hareket hesaplamalarını büyük ölçüde basitleştirir.

3.2 Temel Veri Yapıları

Sistemin durumunu (state) bellekte tutmak için özelleştirilmiş C `struct` ve `enum` yapıları tanımlanmıştır:

Listing 1: Temel Veri Yapılarının Tanımlanması

```
typedef struct {
    int x, y;
} Point;

typedef enum { DIR_UP, DIR_DOWN, DIR_LEFT, DIR_RIGHT } Direction;

typedef struct {
    Point body[MAX_LENGTH];
    int length;
    Direction dir;
} Snake;
```

- **Point:** Kartezyen koordinat sistemindeki X ve Y değerlerini saklar. Yılanın her bir boğumu ve yem nesnesi bu yapı ile temsil edilir.
- **Direction:** Yılanın o anki ilerleme yönünü belirler. Hatalı yön atamalarını önlemek için tamsayılar yerine tip güvenli `enum` kullanılmıştır.
- **Snake:** Yılanın tüm anatomisini barındırır. `body` dizisi, yılanın alabileceği maksimum uzunluk olan 768 (ekrandaki toplam hücre sayısı) kapasitesiyle statik olarak ayrılmıştır.

4 Uygulama ve Algoritma Detayları

4.1 Ana Oyun Döngüsü (The Game Loop)

Projenin kalbini oluşturan ana döngü, `while (running)` yapısı üzerine kuruludur. Modern oyun geliştirme pratiklerine uygun olarak döngü her iterasyonda üç ana işlevi yerine getirir: Girişleri İşleme (Input), Oyun Durumunu Güncelleme (Update) ve Ekranı Çizdirme (Render).

4.2 Kuyruk Kaydırma ve Hareket Algoritması

Yılanın ilerlemesi `snake_move` fonksiyonu aracılığıyla sağlanır. Yılan hareket ettiğinde, tüm vücudun kordinatlarını tek tek hesaplamak yerine performanslı bir "dizi kaydırma" tekniği kullanılır. Dizinin en sonundaki elemandan başlanarak, her boğum bir önceki boğumun koordinatını kopyalar. Son aşamada ise baş kısmı (indeks 0), mevcut hareket yönüne (`Direction`) göre +1 veya -1 birim kaydırılır.

4.3 Zamanlayıcı (Timer) Entegrasyonu

Sistemin farklı CPU hızlarında aynı tepkiyi vermesini sağlamak için delta-zaman mantığı kullanılmıştır. `SDL_GetTicks()` fonksiyonu ile milisaniye cinsinden geçen süre takip edilir. Eğer son hareketin üzerinden 250 milisaniye geçmişse yılan bir adım ilerletilir. Bu sayede yılanın saniyedeki kare hızı (FPS) ekrana çizim hızından bağımsız hale getirilmiştir.

4.4 Çarpışma Tespit Algoritmaları (Collision Detection)

Oyunun kurallarını belirleyen iki ana çarpışma senaryosu mevcuttur:

- 1. Duvar Çarpışması:** Kafa boğumunun x ve y koordinatları, 0 ile ızgara sınırları (`GRID_WIDTH` ve `GRID_HEIGHT`) arasında denetlenir. Sınırın aşılması durumunda oyun biter.

- 2. Öz Çarpışma (Self Collision):** Yılanın kendi etrafında dönüp kendine çarpmasını tespit etmek için bir `for` döngüsü kullanılır. Kafanın (`body[0]`) koordinatları, gövdenin geri kalan tüm elemanlarıyla karşılaştırılır. Eğer herhangi bir gövde elemanı ile kafanın koordinatları tamamen örtüşürse `game_over` bayrağı tetiklenir.

4.5 Rastgele Yem Üretimi (Food Spawning)

Yemlerin ekranda rastgele belirmesi için C'nin standart kütüphanesinden `rand()` fonksiyonu kullanılmıştır. Fonksiyonun her çalıştırmada aynı dizilimi üretmesini engellemek için, oyunun başlangıcında `srand(SDL_GetTicks())` ile işlemcinin çalışma süresi kullanılarak eşsiz bir tohum (seed) değeri oluşturulmuştur.

5 Kullanıcı Arayüzü ve Kaynak Yönetimi

5.1 Oyun Durumu Yönetimi

`game_over` değişkeni 1 değerini aldığı anda sistem duraklatma (pause) durumuna geçer ve ekran temizlenerek Oyun Bitiş arayüzü çizilir. Bu aşamada kullanıcıya sunulan iki seçenek olay döngüsünde (event queue) yakalanır:

- **R (Restart):** Yılanın başlangıç uzunluğu, konumu, skor değeri ve yemin koordinatları `snake_init` fonksiyonu ile sıfırlanarak oyun baştan başlatılır.
- **Q (Quit):** Döngü kırılır ve program sonlandırma aşamasına geçer.

5.2 Bellek Temizliği (Memory Management)

Programın çalışma süresince işletim sisteminden talep edilen bellek alanları, program kapanırken sistem kaynaklarına iade edilir. Bellek sızıntılarını (memory leak) önlemek amacıyla döngü bitiminde sırasıyla şu işlemler gerçekleştirilir:

1. `TTF_CloseFont` ile yüklenen yazı tipleri bellekten atılır.
2. `TTF_Quit` ve `SDL_Quit` ile alt sistemler güvenli bir şekilde kapatılır.
3. `SDL_DestroyRenderer` ve `SDL_DestroyWindow` fonksiyonları ile grafiksel objeler yok edilir.

6 Sonuç ve Değerlendirme

Bu proje çalışması ile C dilinin pointer ve yapı (struct) yönetimi yetenekleri, endüstri standardı olan SDL3 kütüphanesinin düşük seviyeli API'leri ile başarılı bir şekilde entegre edilmiştir. Geliştirme süreci boyunca;

- Sabit bellek tahsisi (static memory allocation) ile array manipölasyonunun performans etkisi,
- İşletim sistemi seviyesindeki donanım olaylarının (klavye kesmeleri) oyun döngüsü içerisinde asenkron işlenmesi,
- Grafıksel yüzeylerin donanım hızlandırmalı dokulara (hardware accelerated textures) dönüştürölmesi,
- Milisaniye hassasiyetinde oyun durumu zamanlaması (game state timing)

gibi kritik yazılım mühendisliğı ve oyun programlama disiplinleri pratik edilmiştir.

6.1 Gelecek Geliştirmeler (Future Work)

Projenin temel mekanikleri kusursuz çalışmakla birlikte, mimari ilerleyen dönemlerde şu özellikleri destekleyecek şekilde genişletilebilir:

- **Dinamik Zorluk Eğrisi:** Yılan yem yedikçe 250ms olan gecikme (delay) süresinin matematiksel bir formülle kademeli olarak düşürölmesi.
- **Veri Kalıcılığı (Persistence):** FILE pointer'ları ve dosya I/O işlemleri kullanılarak en yüksek skorun (High Score) yerel diskte saklanması.
- **İşitsel Geri Bildirim:** SDL3_mixer kütüphanesi entegre edilerek oyun deneyimini zenginleştirecek ses efektlerinin dahil edilmesi.